```python
"""Week 12 DSP + ML demos: PCA/SVM and trees/random forests.

Synthetic audio-feature data keeps the examples self-contained. The same ideas
transfer directly to real MFCC/spectral-feature matrices with shape
(n_clips, n_features).
"""
from __future__ import annotations

import numpy as np
import pandas as pd
from sklearn.decomposition import PCA
from sklearn.ensemble import RandomForestClassifier
from sklearn.inspection import permutation_importance
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

FEATURES = [
    "Spectral centroid", "Zero-crossing rate", "Spectral roll-off",
    "RMS energy", "Spectral flux", "MFCC 1", "MFCC 2", "MFCC 3",
]
CLASSES = ["Kick", "Snare", "Cymbal"]


def make_synthetic_audio_features(seed: int = 42, n_per_class: int = 180) -> tuple[pd.
DataFrame, np.ndarray, np.ndarray]:
    """Create a reproducible audio-like tabular classification data set."""
    rng = np.random.default_rng(seed)
    means = np.array([
        [0.20,0.12,0.25,0.82,0.22,-1.00,0.40,-0.20],
        [0.48,0.48,0.52,0.62,0.72,-0.20,-0.55,0.45],
        [0.82,0.82,0.88,0.46,0.86,0.75,0.20,0.62],
    ])
    base_cov = np.array([
        [1,.45,.72,-.10,.30,.10,.05,.05], [.45,1,.50,-.05,.55,.05,.05,.10],
        [.72,.50,1,-.08,.35,.10,.05,.05], [-.10,-.05,-.08,1,-.15,-.25,.05,.05],
        [.30,.55,.35,-.15,1,.10,-.05,.15], [.10,.05,.10,-.25,.10,1,.25,.10],
        [.05,.05,.05,.05,-.05,.25,1,.20], [.05,.10,.05,.05,.15,.10,.20,1],
    ])
    scales = np.array([.12,.12,.11,.10,.12,.28,.25,.25])
    cov = np.diag(scales) @ base_cov @ np.diag(scales)

    rows = []
    for i, label in enumerate(CLASSES):
        Xi = rng.multivariate_normal(means[i], cov, size=n_per_class)
        rows.extend([(*row, label) for row in Xi])
    df = pd.DataFrame(rows, columns=FEATURES + ["Class"])
    return df, df[FEATURES].to_numpy(), df["Class"].to_numpy()


def demo_pca_svm(X: np.ndarray, y: np.ndarray) -> None:
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.30, random_state=7, stratify=y
    )
    model = Pipeline([
        ("scale", StandardScaler()),
        ("pca", PCA(n_components=0.95)),
        ("svm", SVC(kernel="rbf", C=3.0, gamma="scale")),
    ])
    model.fit(X_train, y_train)
    print("PCA + RBF SVM test accuracy:", round(model.score(X_test, y_test), 3))
    print("PCA components retained:", model.named_steps["pca"].n_components_)


def demo_trees(X: np.ndarray, y: np.ndarray) -> None:
```

```
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.30, random_state=7, stratify=y
    )
    tree = DecisionTreeClassifier(max_depth=4, min_samples_leaf=8, random_state=1)
    forest = RandomForestClassifier(
        n_estimators=300, max_features="sqrt", min_samples_leaf=2,
        oob_score=True, random_state=1, n_jobs=-1,
    )
    tree.fit(X_train, y_train)
    forest.fit(X_train, y_train)
    print("Decision-tree test accuracy:", round(tree.score(X_test, y_test), 3))
    print("Random-forest test accuracy:", round(forest.score(X_test, y_test), 3))
    print("Random-forest OOB accuracy:", round(forest.oob_score_, 3))

    perm = permutation_importance(forest, X_test, y_test, n_repeats=10, random_state=5)
    importance = pd.DataFrame({
        "feature": FEATURES,
        "MDI": forest.feature_importances_,
        "permutation": perm.importances_mean,
    }).sort_values("permutation", ascending=False)
    print("\nFeature importance:\n", importance.to_string(index=False))


if __name__ == "__main__":
    frame, X, y = make_synthetic_audio_features()
    print(frame.groupby("Class")[FEATURES].mean().round(3))
    demo_pca_svm(X, y)
    demo_trees(X, y)

```